

3교시: API 계약과 운영 문서

수업 목표

- OpenAPI/Swagger 개념, endpoint, health, dependency 정보를 MSA 운영 문제로 설명한다.
- 서비스 경계, 네트워크, 설정, 로그, health 중 최소 두 가지 evidence로 정상 상태를 판단한다.
- 장애가 나면 재현, 관찰, 가설, 수정, 재확인 순서로 기록한다.

선행 지식

이미 알고 있어야 할 것	오늘 다시 연결할 것
Docker image/container	MSA의 서비스별 실행 단위
Docker Compose service	서비스 토폴로지와 의존성
port binding, service name DNS	내부 통신과 외부 진입점 구분
logs/status evidence	분산 로그와 장애 분석

50분 흐름

시간	활동	학습 초점	학생 산출
0-5분	문제 상황 제시	왜 단일 앱보다 운영이 어려워지는지 본다.	오늘의 질문 1개
5-15분	핵심 개념 설명	API 계약과 운영 문서의 운영 의미를 정리한다.	용어 note
15-25분	구조/공식 기준 확인	service, dependency, health, logs를 공식 용어와 연결한다.	판단 표
25-40분	실습 또는 데모	Compose 상태와 로그로 실제 증거를 확인한다.	command evidence
40-47분	장애/비용/보안 점검	운영 위험과 복구 기준을 정리한다.	risk note
47-50분	다음 교시 연결	다음 service boundary로 확장한다.	handoff 문장

시작 상황

서비스가 하나일 때는 container가 켜졌는가 , port가 열렸는가 , HTTP 응답이 오는가 를 보면 대부분의 상태를 파악할 수 있다. MSA에서는 같은 질문을 서비스마다 반복해야 하고, 한 서비스의 정상 상태가 전체 사용자 경험의 정상 상태를 보장하지 않는다.

오늘의 주제인 API 계약과 운영 문서 은 이 복잡도를 줄이기 위한 관찰 기준을 만든다. 인프라/DevOps 엔지니어는 내부 구현을 모두 알 필요는 없지만, 어떤 서비스가 어떤 주소로 누구에게 요청하고, 어떤 설정을 필요로 하며, 어디에서 실패 증거를 남기는지는 알아야 한다.

Generated Visual: 3교시 전용 인포그래픽

Week 3 Day 4 Lesson 3 - API 계약과 운영 문서

API 계약은 시스템 간 약속입니다. OpenAPI(=Swagger)로 명확하게 정의하고, 운영 문서로 팀 전체가 일관되게 운영합니다.

1. API 계약이란?

API 제공자와 소비자 간의 명확한 약속(Contract)
어떤 데이터를, 어떤 방식으로 주고받는지 정의
변경 시 버전 관리로 안정성 보장

OpenAPI (v3.x)

Swagger (UI & 문서화)

계약의 주요 대상

endpoint
경로

method
HTTP 메서드

request
요청 구조

response
응답 구조

error code
오류 코드

장점

- 명확한 의사소통으로 오해 감소
- 자동 문서화/테스트/클라이언트 코드 생성 가능
- 변경 영향 파악 및 버전 관리 용이
- 온보딩 및 운영 효율 향상

2. API 명세 예시 (OpenAPI)

주요 엔드포인트 정의				
Endpoint	Method	요청 (Request)	응답 (Response)	설명
/api/health Health	GET	없음	200 OK { "status": "ok", "service": "api", "version": "1.0.0" }	서비스 상태 확인 (Health Check)
/api/users Users	GET	Query page: integer size: integer	200 OK { "items": [...], "total": 123 }	사용자 목록 조회
/api/users Users	POST	Body (JSON) { "name": string, "email": string }	201 Created { "id": string, "name": string, "email": string }	사용자 생성
/api/users/{id} Users	GET	Path id: string	200 OK { "id": string, "name": string, "email": string }	사용자 상세 조회
/api/users/{id} Users	PUT	Path id: string Body (JSON) { "name": string }	200 OK { "id": string, "name": string, "email": string }	사용자 정보 수정
/api/users/{id} Users	DELETE	Path id: string	204 No Content	사용자 삭제

오류 코드 표준

400 Bad Request
요청 파라미터 오류

401 Unauthorized
인증 실패

403 Forbidden
권한 없음

404 Not Found
리소스 없음

409 Conflict
충돌/중복

500 Internal Server Error
서버 내부 오류

Swagger UI
사람이 읽기 쉬운 문서 제공
테스트(try-it-out) 지원

코드/테스트 자동화
클라이언트 SDK 생성
계약 기반 테스트 가능

버전 관리
버전별 문서 관리
하위 호환성 유지

3. 운영 문서 (Operations Document)

서비스 개요

- 목적 및 역할
- 담당팀 / 연락처
- 배출 환경 / 버전

접속 정보

- Base URL
- 네트워크 접근 조건
- 인증 방식 (API Key, JWT 등)

Health Endpoint

- GET /api/health
- 정상 기준 (status: "ok")
- 알람 연동 기준

의존성 (Dependency)

- Database: PostgreSQL
- 외부 API: 결제 서비스
- 메시지 큐: Redis (worker)

운영 가이드

- 주요 설정 값
- 롤백 방법
- 장애 시 대응 절차

변경 이력 (Changelog)

- 버전별 변경 사항
- Deprecated 항목

핵심 원칙

- 계약 우선(Contract First)
- 문서화 자동화(Doc as Code)
- 변경은 버전과 함께(Versioning)
- 관할 가능성(모니터링/알람)과 연계

계약 수립 → 운영까지 흐름

협업 & 설계
API 범위 정의

OpenAPI 작성
계약 명세화

검증 & 테스트
자동화 검증/테스트

배포
계약에 맞게 구현/배포

운영 & 모니터링
문서 기준으로 운영

이 이미지는 Week 3 Day 4 3교시 API 계약과 운영 문서 의 핵심 개념을 세션 전용으로 시각화한 자료다. 다른 교시와 같은 그림을 재사용하지 않고, 이 세션에서 확인할 운영 evidence와 판단 기준에 맞춰 읽는다.

Visual 1: 서비스 토폴로지 읽기

```
graph LR; User[User / Browser] --> Frontend[frontend  
external port 18083]; Frontend --> API[api  
internal port 8080]; API --> Database[(database  
PostgreSQL 5432)]; Worker[worker] --> API; Worker --> Database;
```

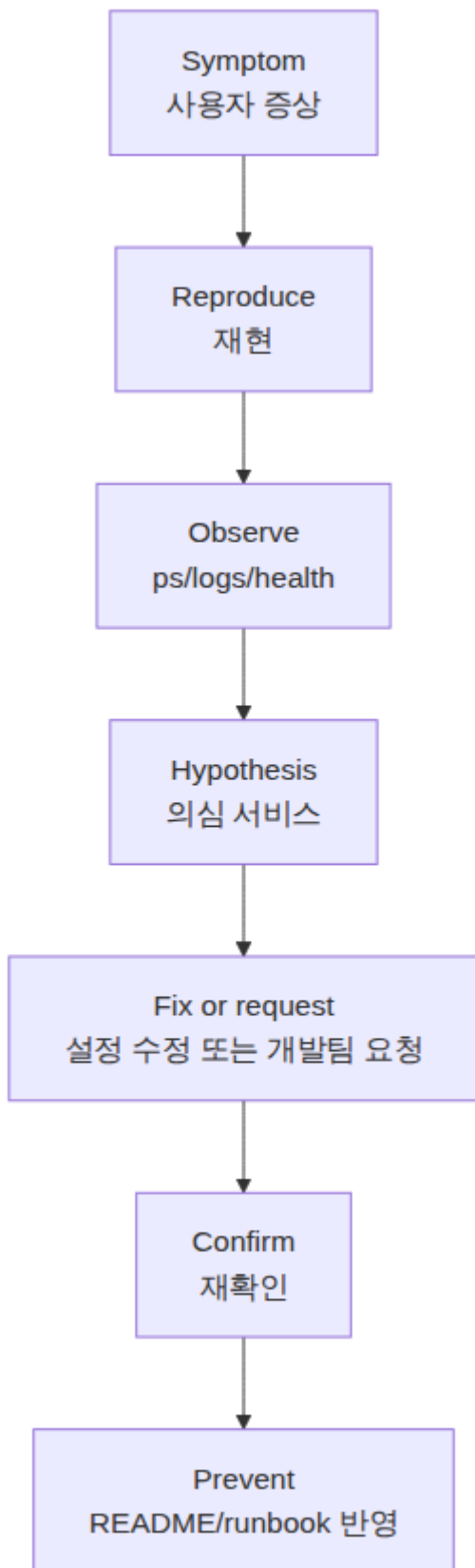
읽는 순서: 사용자는 frontend로 들어오고, frontend는 api로 요청을 넘긴다. api와 worker는 database에 의존한다. 이 그림에서 중요한 것은 박스 개수가 아니라, 장애가 어디에서 어디로 전파될 수 있는지 보는 것이다.

Visual 2: 운영 판단 표

확인 대상	정상 evidence	실패 증상	다음 행동
frontend	browser 접속, HTTP 200	화면은 뜨지만 API 데이터 없음	proxy/API URL 확인
api	/health, /api/status 응답	5xx, timeout, DB unavailable	env, DB host, logs 확인
worker	주기적 처리 로그	처리 정지, 반복 실패	dependency와 retry 주기 확인
database	healthcheck healthy	connection refused	volume, credential, readiness 확인

이 표는 기능 목록이 아니라 장애 분석 순서다. 운영자는 사용자가 본 증상에서 시작해 어느 서비스 evidence를 먼저 볼지 정해야 한다.

Visual 3: 장애 분석 흐름



이 흐름은 Week 1의 RCA와 같다. 달라진 점은 관찰 대상이 하나의 process가 아니라 여러 service의 조합이라는 점이다.

실습 기준

repository root에서 시작한다. Day 1 이후 모든 교시는 같은 실습 앱을 기준으로 진행한다.

```
cd week3/day1/labs/msa-demo
docker compose ps
docker compose logs --tail=40 api
docker compose logs --tail=40 worker
curl -s http://localhost:18083/api/status
```

기대 결과는 서비스 상태가 모두 `running` 또는 `healthy` 에 가깝고, `/api/status` 가 frontend를 통해 api 상태를 JSON으로 돌려주는 것이다. 실패하면 실패 자체를 evidence로 남긴다.

DevOps 원칙 연결

관점	오늘의 연결
비용 절감	로컬 Compose로 topology를 먼저 검증해 cloud 리소스 생성 전 오류를 줄인다.
개발/배포 효율	서비스별 실행 계약을 문서화해 개발팀과 운영팀의 왕복 질문을 줄인다.
관리 효율	health, logs, dependency map을 표준화해 장애 triage 시간을 줄인다.

흔한 오해

오해	바로잡기
MSA는 서비스를 많이 나누면 자동으로 좋아진다.	서비스 경계가 운영 비용보다 큰 이익을 줄 때 선택한다.
frontend가 보이면 전체 시스템이 정상이다.	API나 DB 장애가 숨어 있을 수 있으므로 service별 health를 봐야 한다.
depends_on이면 DB 준비까지 보장된다.	실행 순서와 readiness는 다르다. healthcheck와 retry가 필요하다.
로그를 보면 다 알 수 있다.	여러 서비스 로그를 연결할 request id/correlation id가 없으면 원인 추적이 느려진다.

Evidence Note 양식

```
# Week 3 Evidence - API 계약과 운영 문서

## Service checked
- Service:
- Command:
- Expected:
- Actual:

## Dependency
- Upstream:
- Downstream:
- Config/env used:

## Failure or risk
- Symptom:
- Observed evidence:
```

- Fix or request:
- Recheck result:

평가 기준

기준	통과 조건
개념 이해	API 계약과 운영 문서를 서비스 경계/의존성/운영 증거 중 하나와 연결했다.
실행 증거	Compose 상태, 로그, HTTP 응답 중 하나 이상을 기록했다.
장애 분석	증상과 의심 서비스를 구분했다.
협업 기준	개발팀에 요청할 정보 또는 운영 문서에 남길 정보를 적었다.
다음 연결	Kubernetes에서 어떤 리소스로 이어질지 한 문장으로 설명했다.

공식/학술 근거 링크

- AWS Microservices on AWS, <https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices.html> - MSA를 여러 서비스와 API 기반 구조로 설명하는 공식 기준이다.
- Docker Compose Docs, <https://docs.docker.com/compose/> - 여러 service를 파일로 실행하고 관리하는 기준이다.
- Google SRE Book: Addressing Cascading Failures, <https://sre.google/sre-book/addressing-cascading-failures/> - 한 서비스 장애가 다른 서비스로 퍼질 수 있음을 설명하는 근거다.

다음 연결

다음 교시는 같은 실습 앱에서 다른 service boundary를 본다. 오늘 만든 evidence note는 서비스 목록, 의존성, health, logs를 누적하는 운영 문서가 된다.